Claude Code 深度使用手册

从入门到精通的完整指南

版本 1.0 | 2025 年 12 月

第一章 Claude Code 概述

1.1 什么是 Claude Code

Claude Code 是 Anthropic 推出的代理式编码工具，它直接在终端中运行，能够理解你的代码库，并通过自然语言命令帮助你更快地编写代码。它可以执行日常任务、解释复杂代码、处理 Git 工作流程。

1.2 核心特性

- **终端原生:** 在你熟悉的工作环境中运行，无需切换到其他 IDE 或聊天窗口
- **代码库感知:** 自动理解整个项目结构，无需手动选择上下文
- **直接执行:** 可以直接编辑文件、运行命令、创建提交
- **Unix 哲学:** 可组合、可脚本化，支持管道操作
- **企业级:** 支持 Claude API，可在 AWS 或 GCP 上托管

1.3 适用场景

场景	描述
功能开发	用自然语言描述需求，Claude Code 制定计划、编写代码并确保其正常工作
Bug 修复	描述 bug 或粘贴错误信息，自动分析代码库、定位问题并实现修复
代码导航	询问任何关于代码库的问题，获得深思熟虑的答案
重构优化	重构代码、提升性能、添加测试覆盖
文档编写	自动生成 README、API 文档、注释

第二章 安装与配置

2.1 系统要求

- Node.js 18 或更高版本（使用 npm 安装时需要）
- macOS、Linux 或 Windows（通过 WSL 或 PowerShell）
- Claude Pro/Max 订阅 或 Anthropic API 密钥

2.2 安装方法

方法一：官方脚本安装（推荐）

macOS / Linux:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

方法二：Homebrew 安装（macOS）

```
brew install --cask claude-code
```

方法三：npm 全局安装

```
npm install -g @anthropic-ai/claude-code
```

2.3 认证配置

首次运行时，Claude Code 会提示你进行认证。你有两种选择：

1. **浏览器认证:** 如果你有 Claude Pro/Max 订阅，可以直接通过浏览器登录
2. **API 密钥:** 设置 ANTHROPIC_API_KEY 环境变量

```
export ANTHROPIC_API_KEY="your-api-key-here"
```

è 提示: 将上述命令添加到 ~/.bashrc 或 ~/.zshrc 中以持久化配置

第三章 基础使用

3.1 启动 Claude Code

进入项目目录并运行 claude 命令即可启动交互式会话：

```
cd your-project claude
```

3.2 常用启动参数

参数	说明	示例
-p, --print	打印模式，执行单次查询后退出	claude -p "解释这段代码"
--dangerously-skip-permissions	跳过所有权限确认	claude --dangerously-skip-permissions
--model	指定使用的模型	claude --model claude-sonnet-4-20250514
--output-format	输出格式（text/json/stream-json）	claude -p "query" --output-format json
--system-prompt	自定义系统提示词	claude --system-prompt "你是代码审查专家"
--agents	定义子代理（JSON 格式）	claude --agents '{...}'
--mcp-debug	启用 MCP 调试模式	claude --mcp-debug
--debug	启用调试模式，显示详细信息	claude --debug

3.3 交互式命令

快捷键

快捷键	功能
Esc	中断当前操作
Esc + Esc	打开撤销菜单（回退更改）
Ctrl + R	显示完整输出/上下文
Ctrl + V	粘贴图片
Shift + Tab	自动接受模式（YOLO 模式）
Shift + Tab + Tab	计划模式
!	Bash 模式前缀（直接运行命令）
@	提及文件/目录
\	换行（反斜杠 + Enter）

斜杠命令

命令	功能
/help	显示帮助信息和所有可用命令
/clear	清除对话历史
/compact	压缩上下文（保留摘要）
/config	查看和修改配置
/cost	显示当前会话的 token 使用量和费用
/doctor	诊断安装和配置问题
/hooks	管理钩子配置
/agents	管理子代理
/mcp	管理 MCP 服务器连接
/plugin	管理插件
/bug	报告 bug
/install-github-app	安装 GitHub 应用以自动审查 PR

第四章 CLAUDE.md 配置文件

4.1 什么是 CLAUDE.md

CLAUDE.md 是 Claude Code 的记忆文件，用于存储项目上下文、编码规范、常用命令等信息。每次启动会话时，Claude 都会自动加载这些信息，无需重复说明。

4.2 文件层级

CLAUDE.md 支持层级化配置，优先级从高到低：

1. **嵌套目录:** 项目子目录中的 CLAUDE.md（最高优先级）
2. **项目级:** .claude/CLAUDE.md 或项目根目录的 CLAUDE.md
3. **用户级:** ~/.claude/CLAUDE.md（适用于所有项目）

4.3 推荐内容结构

```
# 项目概述 ## 架构 – **前端**: Next.js 14 + TypeScript – **后端**: Node.js + Express – **数据库** PostgreSQL + Prisma ORM – **部署**: Vercel (前端), Railway (后端) ## 关键命令 – `npm run dev` – 启动开发服务器 – `npm run build` – 生产构建 – `npm run test` – 运行测试 – `npm run lint` – 代码检查 ## 编码规范 – 所有新代码使用 TypeScript 严格模式 – 使用函数式组件和 Hooks – 变量名必须具有描述性 – 公共函数需要 JSDoc 注释 – 组件不超过 200 行 ## 测试要求 – 使用 Jest + React Testing Library – 代码覆盖率目标 80% – 包含边界情况测试
```

è 提示: 让 Claude 自动生成 CLAUDE.md：输入 "帮我创建一个 CLAUDE.md 文件，分析当前项目结构"

第五章 权限与安全

5.1 权限配置文件

权限配置存储在 settings.json 文件中，支持以下位置（优先级从高到低）：

1. 企业级: /etc/claude-code/managed-settings.json
2. 用户级: ~/.claude/settings.json
3. 项目共享: .claude/settings.json
4. 项目本地: .claude/settings.local.json（git 忽略）

5.2 权限规则语法

```
{ "permissions": { "allow": [ "Bash(npm run *)", "Bash(git status)", "Bash(git diff)", "Bash(git add *)", "Bash(git commit *)" ], "ask": [ "Bash(git push:*)", "Bash(npm install *)" ], "deny": [ "Read(./.env*)", "Read(./secrets/**)", "Bash(rm -rf:*)", "Bash(curl:*)" ] } }
```

5.3 权限处理顺序

权限检查按以下顺序处理：

PreToolUse Hook → Deny 规则 → Allow 规则 → Ask 规则 → 权限模式检查 → canUseTool 回调 → PostToolUse Hook

> 注意: deny 规则的优先级高于 allow 规则，始终会被执行

第六章 Hooks 系统

6.1 什么是 Hooks

Hooks 是 Claude Code 的自动化工作流系统，允许你在特定事件发生时自动执行 shell 命令。与提示词指令不同，Hooks 是确定性的，无论 Claude 的决策如何都会执行。

6.2 Hook 事件类型

事件	触发时机	用途
UserPromptSubmit	用户提交提示词后	提示词验证、上下文注入、安全过滤
PreToolUse	工具执行前	阻止危险命令、自动审批、日志记录
PostToolUse	工具执行后	自动格式化、测试运行、通知
PermissionRequest	显示权限对话框时	自动授权或拒绝
Notification	发送通知时	自定义通知方式
Stop	会话结束时	生成摘要、保存日志
SubagentStop	子代理完成时	后续处理
PreCompact	压缩操作前	备份上下文
SessionStart	会话开始时	加载开发上下文

6.3 Hook 配置示例

自动格式化

```
{ "hooks": { "PostToolUse": [{ "matcher": "Edit|Write", "hooks": [{ "type": "command", "command": "jq -r '.tool_input.file_path' | { read fp; if echo \"$fp\" | grep -q '\\.ts$'; then npx prettier --write \"$fp\"; fi; }" }] }] } }
```

命令日志记录

```
{ "hooks": { "PreToolUse": [{ "matcher": "Bash", "hooks": [{ "type": "command", "command": "jq -r '\"\\(.tool_input.command) - \\(.tool_input.description // \\\"No description\\\")\"' >> ~/.claude/bash-log.txt" }] }] } }
```

6.4 Hook 返回值控制

返回码	行为	说明
0	成功	Hook 执行成功，stdout 在转录模式下显示
2	阻塞错误	stderr 自动反馈给 Claude
其他	错误	Hook 执行失败

第七章 自定义命令

7.1 创建自定义命令

自定义命令是 Markdown 文件形式的提示词模板，存储在 commands 目录中：

- 项目级: .claude/commands/ （团队共享）
- 用户级: ~/.claude/commands/ （个人全局）

7.2 命令示例

代码审查命令

文件: .claude/commands/review.md

```
# 代码审查 请对以下代码进行全面审查： 1. **安全性**: 检查潜在漏洞 2. **性能**: 识别优化机会 3. **可维护性**: 评估代码结构和清晰度 4. **测试**: 建议测试覆盖改进 5. **最佳实践**: 确保符合项目规范 重点关注: $ARGUMENTS 请提供具体且可操作的反馈。
```

修复 Issue 命令

文件: .claude/commands/fix-issue.md

```
# 修复 GitHub Issue 请分析并修复 GitHub Issue: $ARGUMENTS 步骤： 1. 使用 `gh issue view` 获取 issue 详情 2. 理解问题描述和根本原因 3. 搜索代码库找到相关文件 4. 实现必要的修改 5. 编写并运行测试验证修复 6. 确保代码通过 lint 和类型检查 7. 创建描述性提交信息 8. 推送并创建 PR 使用 GitHub CLI (`gh`) 执行所有 GitHub 相关任务。
```

7.3 使用命令

在 Claude Code 会话中使用斜杠调用命令：

```
/project:review src/utils/auth.ts /project:fix-issue 1234
```

第八章 子代理 (Subagents)

8.1 什么是子代理

子代理是专门化的 AI 助手，可以被委派处理特定类型的任务。每个子代理拥有独立的上下文窗口、自定义系统提示词和工具访问权限，适合处理复杂问题时保持主对话的专注性。

8.2 子代理优势

- **上下文隔离:** 独立的上下文窗口，防止主对话被污染
- **专业化:** 针对特定领域的详细指令，提高成功率
- **并行执行:** 可以同时运行多个子代理处理不同任务
- **工具控制:** 为每个子代理分配特定的工具访问权限

8.3 创建子代理

子代理配置文件存储在 agents 目录中：

- 项目级: .claude/agents/ （最高优先级）
- 用户级: ~/.claude/agents/

子代理配置示例

文件: .claude/agents/code-reviewer.md

```
--- name: code-reviewer description: 专业代码审查员。在代码更改后主动使用。 tools: Read, Grep, Glob, Bash model: inherit --- 你是一位资深代码审查员，专注于： - 代码质量和可读性 - 安全漏洞检测 - 性能优化建议 - 最佳实践合规性 审查时，提供具体的行号和改进建议。
```

8.4 使用子代理

Claude 会自动根据任务选择合适的子代理，也可以显式调用：

```
# 显式调用 "使用 code-reviewer 代理审查 src/api/auth.ts" # 触发子代理使用 "使用子代理验证这个实现" "用多个子代理分析这个问题"
```

第九章 MCP 集成

9.1 什么是 MCP

MCP (Model Context Protocol) 是 Anthropic 提出的协议，用于连接 AI 系统与外部工具。它允许 Claude Code 访问专业化的实时信息和工具，如数据库、API、文档系统等。

9.2 添加 MCP 服务器

HTTP 服务器（推荐）

```
# 基本语法 claude mcp add --transport http <name> <url>  # 示例：连接 Notion claude mcp add --transport http notion https://mcp.notion.com/mcp  # 带认证 claude mcp add --transport http secure-api https://api.example.com/mcp \  --header "Authorization: Bearer your-token"
```

本地 Stdio 服务器

```
# 基本语法 claude mcp add <name> -- <command> [args...]  # 示例：添加 Context7（文档查询） claude mcp add context7 -- npx -y @upstash/context7-mcp  # 示例：添加 Brave Search claude mcp add brave-search -s user -- env BRAVE_API_KEY=your-key \ npx -y @modelcontextprotocol/server-brave-search
```

9.3 MCP 配置作用域

作用域	配置位置	适用场景
本地	.mcp.json	当前目录的临时配置
项目	.claude/.mcp.json	团队共享，签入版本控制
用户	~/.claude.json	个人全局配置

9.4 常用 MCP 服务器

服务器	用途	安装命令
Context7	查询 OSS 库文档	npx -y @upstash/context7-mcp
Brave Search	网页搜索	npx -y @modelcontextprotocol/server-brave-search
Puppeteer	浏览器自动化/截图	npx -y @anthropic/mcp-puppeteer
Sentry	错误监控集成	npx -y @sentry/mcp-server
GitHub	GitHub 操作	npx -y @anthropic/mcp-github
PostgreSQL	数据库访问	npx -y @anthropic/mcp-postgres

第十章 Plugins 系统

10.1 什么是 Plugins

Plugins 是打包和分享 Claude Code 自定义配置的轻量级方式，可以包含斜杠命令、子代理、MCP 服务器和 Hooks 的任意组合。通过 /plugin 命令安装和管理。

10.2 插件管理命令

命令	功能
/plugin	打开插件管理界面
/plugin marketplace add <url>	添加插件市场
/plugin install <name>@<marketplace>	从市场安装插件
/plugin enable <name>@<marketplace>	启用已禁用的插件
/plugin disable <name>@<marketplace>	禁用插件（不卸载）
/plugin uninstall <name>@<marketplace>	完全移除插件

10.3 插件使用场景

- **执行标准:** 工程负责人通过插件确保团队遵循特定的 hooks 进行代码审查或测试
- **用户支持:** 开源维护者提供斜杠命令帮助开发者正确使用其包
- **工作流共享:** 开发者将调试、部署、测试工作流打包分享
- **工具连接:** 团队使用插件快速连接内部工具和数据源

第十一章 无头模式与自动化

11.1 无头模式概述

无头模式 (Headless Mode) 用于非交互式场景，如 CI/CD、预提交钩子、构建脚本和自动化任务。使用 -p 标志启用。

11.2 基本用法

```
# 单次查询 claude -p "分析 src/api 目录的代码质量"  # JSON 输出 claude -p "列出所有 TODO 注释" --output-format json  # 流式 JSON 输出 claude -p "重构这个文件" --output-format stream-json  # 管道输入 cat error.log | claude -p "分析这些错误并提供解决方案" git diff | claude -p "审查这些更改"
```

11.3 CI/CD 集成示例

GitHub Actions - 自动代码审查

```
name: Claude Code Review on: [pull_request] jobs: review: runs-on: ubuntu-latest steps: - uses: actions/checkout@v4 - name: Install Claude Code run: curl -fsSL https://claude.ai/install.sh | bash - name: Review Changes env: ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }} run: | git diff origin/main | claude -p "审查这些更改，关注安全和性能" --output-format json > review.json
```

Git Pre-commit Hook

```
#!/bin/bash # .git/hooks/pre-commit STAGED_FILES=$(git diff --cached --name-only --diff-filter=ACM | grep -E '\.(ts|tsx|js|jsx)$') if [ -n "$STAGED_FILES" ]; then echo "Running Claude Code review on staged files..." for file in $STAGED_FILES; do result=$(claude -p "检查 $file 是否有明显的 bug 或安全问题，只输出 PASS 或问题描述") if [[ ! "$result" =~ ^PASS ]]; then echo "Issues found in $file:" echo "$result" exit 1 fi done fi
```

第十二章 最佳实践

12.1 提示词技巧

具体明确

Claude 的成功率随着指令的具体程度显著提高。给出清晰的方向可以减少后续修正。

差	好
修复这个 bug	修复 auth.ts 第 42 行的空指针异常，添加 null 检查
写个测试	为 UserService.createUser() 编写单元测试，覆盖成功、用户已存在、邮箱无效三种情况
优化性能	将 getUsers() 的数据库查询从 N+1 改为 JOIN，预期将响应时间从 500ms 降至 50ms

使用思考模式

对于复杂问题，使用 "think" 关键词触发扩展思考模式：

```
"think" < "think hard" < "think harder" < "ultrathink"
```

利用视觉输入

Claude Code 支持图片输入，可用于：

- UI 设计稿作为实现参考
- 截图用于调试和分析
- 图表用于理解业务逻辑

è 提示: macOS 快捷键：cmd+ctrl+shift+4 截图到剪贴板，ctrl+v 粘贴到 Claude

12.2 工作流程建议

1. **先读后写:** 让 Claude 先阅读相关文件，明确不要写代码，再制定计划
2. **使用子代理:** 复杂问题早期使用子代理验证细节，保持主上下文清晰
3. **提供目标:** 给出可迭代的目标（测试用例、视觉稿），让 Claude 自主验证
4. **及时提交:** 完成一个功能点后立即让 Claude 提交，保持原子性更改
5. **定期压缩:** 长会话使用 /compact 压缩上下文，保持响应质量

12.3 安全建议

- 使用权限规则限制敏感文件访问（.env、secrets 等）
- 生产环境避免使用 --dangerously-skip-permissions
- 审查 Hooks 配置，它们以你的凭证执行
- 谨慎添加第三方 MCP 服务器，注意潜在的工具中毒风险
- CI/CD 中使用专用的 API 密钥并限制权限

第十三章 常见问题排查

13.1 诊断命令

```
# 运行诊断 /doctor  # 调试模式启动 claude --debug claude -d
```

13.2 常见问题

问题	解决方案
认证失败	检查 ANTHROPIC_API_KEY 环境变量或重新运行 /login
MCP 服务器连接失败	使用 --mcp-debug 启动，检查配置和网络
Hooks 不生效	检查 /hooks 菜单确认配置，直接编辑 settings 需要审核
上下文过长	使用 /compact 压缩，或开启新会话
权限提示过多	配置 settings.json 的 allow 规则或使用 Shift+Tab
命令找不到	确保 Claude Code 在 PATH 中，或重新安装

13.3 获取帮助

- **官方文档:** https://docs.anthropic.com/en/docs/claude-code
- **报告 Bug:** 在 Claude Code 中使用 /bug 命令
- **GitHub:** https://github.com/anthropics/claude-code
- **Discord 社区:** Claude Developers Discord

附录 A：配置文件速查

文件/目录	位置	用途
CLAUDE.md	项目根目录/.claude/	项目上下文和规范
settings.json	~/.claude/ 或 .claude/	权限和 Hooks 配置
settings.local.json	.claude/	本地配置（git 忽略）
.mcp.json	项目根目录/.claude/	MCP 服务器配置
commands/	~/.claude/ 或 .claude/	自定义斜杠命令
agents/	~/.claude/ 或 .claude/	子代理定义

附录 B：工具权限列表

工具	说明	权限语法示例
Read	读取文件	Read(.env*), Read(src/**)
Write	创建文件	Write(*.ts), Write(src/**)
Edit	编辑文件	Edit(*.py), Edit(tests/**)
Bash	执行命令	Bash(npm *), Bash(git status)
WebFetch	获取网页	WebFetch
WebSearch	网页搜索	WebSearch
Task	子代理任务	Task

附录 C：模型选项

模型	标识符	特点
Claude Sonnet 4	claude-sonnet-4-20250514	默认模型，平衡性能和速度
Claude Opus 4.5	claude-opus-4-5-20251101	最强大，适合复杂任务
Claude Haiku 4.5	claude-haiku-4-5-20251001	最快速，适合简单任务

结语

Claude Code 不仅仅是一个编码助手，它是一个完整的 AI 代理框架。通过合理配置 CLAUDE.md、Hooks、子代理和 MCP，你可以构建高度自动化的开发工作流，显著提升编码效率。

建议从简单的配置开始，逐步探索高级功能。记住，最好的工具是被充分利用的工具——花时间了解 Claude Code 的能力，会在未来节省大量时间。

祝你编码愉快！

— 完 —